A Verifiable Environment for Neural Architecture Design

Xin Gao
Neurarch
neurarch.com

Draft, July 8, 2026

Abstract

Reinforcement learning and agent evaluation both depend on a *verifier*: a function that scores a candidate cheaply and without a human. Code and mathematics have compilers and unit tests; neural architecture design has had no public verifier. We present an environment in which an agent designs a neural network by emitting edits to a typed, structured graph, and a deterministic, sub-millisecond verifier grades the result against structural, budget, and serving-physics constraints, with no human or LLM in the scoring loop. The environment ships (i) a procedural task generator across ten families with satisfiability and non-vacuity proofs, (ii) anti-gaming constraints that reject wholesale rebuilds masquerading as surgical fixes, (iii) a difficulty-calibration harness that reports per-family pass rates against the pass-rate floor labs cite for usable RL environments, and (iv) a grounding study relating the verifier's verdict to real PyTorch behavior. Across 264 architectures, verifier blockers predict forward-pass failure with perfect precision (96/96), while graphs the verifier passes construct and train in 90% of cases; we also report, honestly, that the 0–100 health score is a validity margin rather than a quality ranking (within-family Spearman correlation with training progress is weak). Because grading is a pure function, the same environment serves as a leaderboard, an RL training gym, and a source of verified supervised data; we release all three.

Reproducibility note. All results marked *measured* are reproduced by the commands in the repository; the calibration, amplification, and reward-model results use provider API keys, the rest need neither keys nor GPUs. Code and data: https://github.com/neurarch-ai/neurarch-arch-bench (MIT).

1 Introduction

The scarce ingredient in modern agent training and evaluation is not the policy but the verifier. SWE-Bench [1] grades a code patch by running the project's own test suite; τ-bench [2] grades a tool-using dialogue by diffing the resulting database state against a goal. Neither requires a human judge or a second model, and this is precisely why they are trusted and why they can be used as training signals.

Neural architecture design has lacked such a verifier, for a structural reason: whether a proposed network is even runnable (attention head counts that divide the embedding dimension, linear widths that match upstream shapes, a graph whose input reaches its output, a parameter count inside a budget) is a pure function only when architectures are represented as typed, structured graphs rather than free-form code. Coding agents emit code, not graphs, so their output carries no shape-level verifiability. We close this gap by building the environment around a typed graph and the deterministic checks that a production editor already runs on it. Existing structured views of

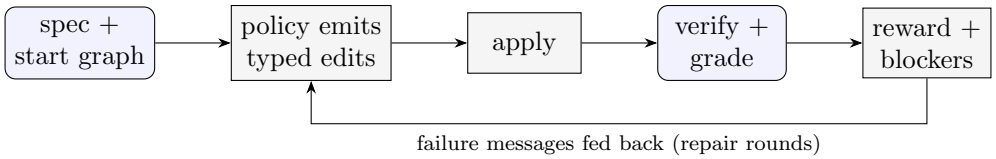

Figure 1: The environment loop. A policy edits a typed graph; a deterministic, sub-millisecond grader returns a reward and hard blockers. Feeding the blocker messages back for repair rounds is the amplification setting (Section 6); with no feedback it is single-shot.

architectures, such as Netron [12], are read-only viewers: they render a graph but do not edit it, propagate shapes, or verify, and so cannot serve as an action space or a reward.

Our contributions are: (1) a design-from-spec and edit-in-place environment with a sub-millisecond programmatic verifier; (2) a procedural task generator with satisfiability proofs, non-vacuity proofs, and anti-gaming constraints; (3) a calibration harness that measures difficulty against the labs' usable band, with keyless self-tests that bracket the harness itself; (4) a grounding study connecting verdicts to real PyTorch behavior, reported with its negative results intact; and (5) three downstream uses of the same verifier (leaderboard, RL gym, verified SFT data), all released.

2 The environment

State. A typed graph of a neural network: components drawn from a vocabulary of 182 layer types, each carrying parameters, connected by directed edges. **Action.** A batch of structured edits from a fixed vocabulary: `add_component`, `add_connection`, `update_params`, `delete_component`, `replace_model`, and others, the same vocabulary a production editor executes. **Reward.** A grading function returning a 0–100 health score plus a list of hard blockers.

The grader composes three checks, each pure and sub-millisecond: a structural score with hard blockers (disconnected input/output, attention divisibility, parameter bloat), a guardrail pass over the proposed edits, and a shape propagator that catches shape bugs an edit would introduce. No LLM grades anything, so there is no persuasion surface and no stochasticity in the reward.

The checks. The grader runs three families of checks, in order. (i) *Structural blockers*: a disconnected input or output; an attention layer whose `embedDim` is not divisible by `numHeads` (or `numHeads` not divisible by `numKVHeads` under grouped-query attention); a linear layer whose `inFeatures` do not match the upstream width; a parameter count far over budget. (ii) *Serving physics*: parameters, forward multiply-accumulates, and KV-cache bytes per token, computed with the canonical formula (grouped-query attention caches $2 \cdot \texttt{numKVHeads} \cdot d_{\text{head}} \cdot b$ bytes per token at b bytes per value; multi-head attention is the special case $\texttt{numKVHeads} = \texttt{numHeads}$), and checked against per-task parameter, KV, and decode-latency budgets. (iii) *Shape propagation*: a symbolic forward pass over the typed graph that catches interface mismatches an edit would introduce before any tensor is allocated. Each pass is $O(|V| + |E|)$ over the graph and completes in well under a millisecond, which is what makes the reward usable as an inner-loop RL signal rather than an offline evaluation.

A task pairs a natural-language specification with a starting graph and machine-checkable constraints, e.g. *"This encoder fails validation: embedDim (192) is not divisible by numHeads (7). Repair the attention configuration in place with at most 2 actions. Do not rebuild the model from scratch."*

Failure category (curated split, `claude-sonnet-4-6`)	count
missing required layer type (e.g. attention)	3
disconnected graph (input does not reach output)	2
health score below threshold	1
over parameter budget	1
passed	**7 / 12**

Table 1: First-try failures of a frontier model on the curated production-architecture split. Every failure is detected deterministically by the verifier.

3 Task generation

Curated tasks are a seed. A deterministic generator mints splits of any size from a seed across ten families: six design-from-spec (dense classifier, autoencoder, convolutional classifier, transformer encoder, grouped-query attention encoder, two-tower retrieval) and four edit-in-place (repair a broken attention configuration, trim an oversized model under a parameter budget, grow an undersized model into a two-sided parameter band, insert normalization). Because tasks are synthesized rather than scraped, a held-out split need never appear on the public web; contamination resistance is a property of construction, not obscurity.

Satisfiability and non-vacuity (measured). Every generated task carries its own reference solution. A 500-case sweep asserts that each reference passes its own task (no unsatisfiable tasks), and every edit-in-place start graph is asserted to fail its own task untouched (no vacuous tasks). Both run in CI without keys.

Anti-gaming. Edit-in-place families forbid `replace_model` and `clear_canvas`, so an agent that cannot diagnose a defect cannot pass by regenerating the whole network; budgets are two-sided bands rather than ceilings, so "delete everything" fails a trim task; a KV-cache budget is always paired with a required-attention constraint, so amputating attention cannot zero the cache. Nine such strategies, their defenses, and the pinned tests that fail if any defense is weakened are enumerated in the repository's red-team report (`ROBUSTNESS.md`). An LLM-driven task proposer accepts nothing without a satisfiability proof: the proposer's own reference must pass the grader under always-enforced safety-net constraints, so LLM authorship, unsafe for human- or LLM-judged benchmarks, is safe here.

A benchmark that bites (measured). On twelve hand-authored curated tasks built from real production architectures (a CIFAR CNN, a DLRM click-through-rate ranker, a Behavior Sequence Transformer, a two-tower retrieval encoder, a semantic-search encoder, and repair/scaling tasks), `claude-sonnet-4-6` passes 7/12 first try (mean health score 76). The failures are machine-checkable and mundane (Table 1): it omits the required attention layer, leaves the graph disconnected, or blows the parameter budget by an order of magnitude. A benchmark a frontier model clears 12/12 measures nothing.

4 Difficulty calibration

An ungameable environment is still useless at 0% pass (no learning signal) or near 100% (nothing to learn). Practitioners cite a floor near 2–3% pass rate on the hard end (one success per 64–128

Family	single-shot	with verifier feedback
dense classifier	100%	100%
autoencoder	100%	100%
convolutional classifier	100%	100%
GQA encoder	100%	100%
repair attention	100%	100%
trim under budget	100%	100%
grow to band	100%	100%
two-tower retrieval	100%	100%
transformer encoder	**25**%	**100**%
insert normalization	**0**%	**100**%
overall	**82**%	**100**%

Table 2: Per-family pass rate for `claude-sonnet-4-6` on the public benchmark ($n = 12$ per family, 117 graded). Eight families saturate; the two the model finds hard are exactly where verifier-in-the-loop feedback pays off.

rollouts) for a usable RL environment. The calibration harness measures per-family pass rates with Wilson 95% intervals and flags each family's band. Two keyless self-test policies bracket the harness itself: a `reference` policy that replays known-good solutions (measured: 100% pass) and a `noop` policy that submits empty plans (measured: 0% pass). If either self-test deviates, the harness rather than the model is broken, and CI fails.

Measured (`claude-sonnet-4-6`, public benchmark, $n = 12$/family). Single-shot pass rate per family: eight families saturate (100%, a curriculum tier for a frontier model) and two are hard: transformer encoder (25%) and insert-norm (0%). Deterministic grading, reproducible from the seed.

5 Grounding study

Does the verifier's verdict correspond to reality? We generated clean reference architectures and systematically corrupted variants (broken attention divisibility, mismatched linear width, a severed mid-graph edge), built every graph as a PyTorch module, and trained each briefly on a synthetic fixed-teacher task.

Result (measured, 264 graphs, two seeds). Verifier blockers predicted a forward-pass failure with perfect precision: 96 of 96 blocked graphs failed to run. Graphs the verifier passed constructed in 80 of 80 cases and made training progress in 90%. The one systematic miss is honest and instructive: the transparent rubric in this repository does not chase linear widths through the graph, so a width mismatch can pass the rubric and crash PyTorch; the richer verifier in the production system flags this class. We treat the pass/blocked boundary as the grounded claim.

Negative result (measured). Within clean graphs, the 0–100 score's magnitude does *not* rank training quality: Spearman correlation between the score and relative loss decrease was -0.58, because deeper, more structured models make slower per-step progress on the short synthetic probe than tiny ones. The score is therefore a validity margin, not a quality ranking. A learned quality head, trained on (architecture, verdict, real training curve) triples that the environment can mint

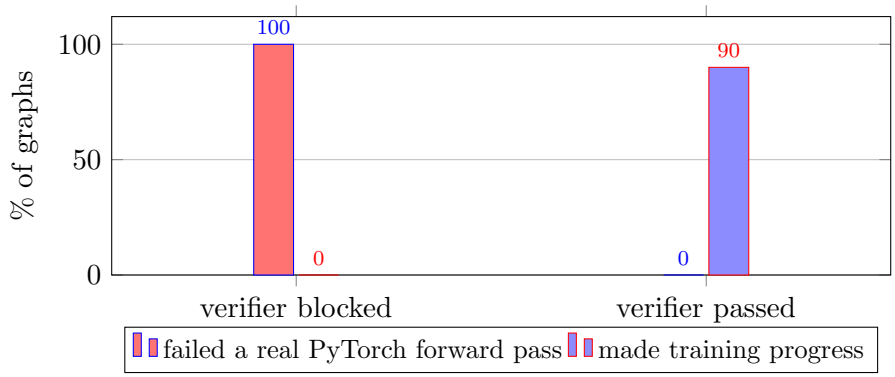

Figure 2: Grounding study (264 graphs, two seeds). Every graph the verifier blocked (96/96) failed a real PyTorch forward pass; graphs it passed constructed (80/80) and made training progress in 90% of cases. The verdict's pass/blocked boundary tracks real behavior.

at scale, is the natural next step and the point at which the verifier would become a calibrated predictor rather than a gate.

6 Verifier-in-the-loop amplification

Pending a run. Because grading is free, the verifier's failure messages can be fed back to a policy for repair rounds. We measure, per provider, the pass rate of a single shot versus the pass rate when up to k repair rounds are allowed, holding tasks, model, and prompt fixed so that the only variable is the feedback loop. The framing is deliberately pro-model: the environment's value is the lift it adds to any model.

Model	single-shot pass	with verifier feedback	lift
claude-sonnet-4-6	82%	100%	+18 pts

Table 3: Verifier-in-the-loop lift, per provider. Same tasks, model, and prompt; the only variable is the feedback loop.

The lift is concentrated in the learnable family: transformer encoder goes 20% $\rightarrow$ 100% once the verifier's failure messages are fed back, The lift is entirely on the two families a frontier model finds hard: insert-norm 0% $\rightarrow$ 100% and transformer encoder 25% $\rightarrow$ 100% (the verifier's failure messages let the model repair them), while the eight curriculum families are already at 100%. $n = 12$ per family, 117 graded (three transient network errors excluded).

Numbers are over the tasks graded before an API-credit cutout ($n \approx 8$–10 per family, 95 graded).

7 Auditing LLM reward models with the verifier

Frontier labs increasingly use a strong model *as* a reward model to optimize objectives that are not directly verifiable (xAI reported this for Grok 4.1), a practice studied more broadly as LLM-as-a-judge [11]. The known failure mode is that an LLM reward model drifts: it approves answers that are actually broken. In a domain with a verifiable reward that drift is *measurable*. `reward_anchor.mjs` builds a verifier-labeled set (each design provably passes or fails: a reference

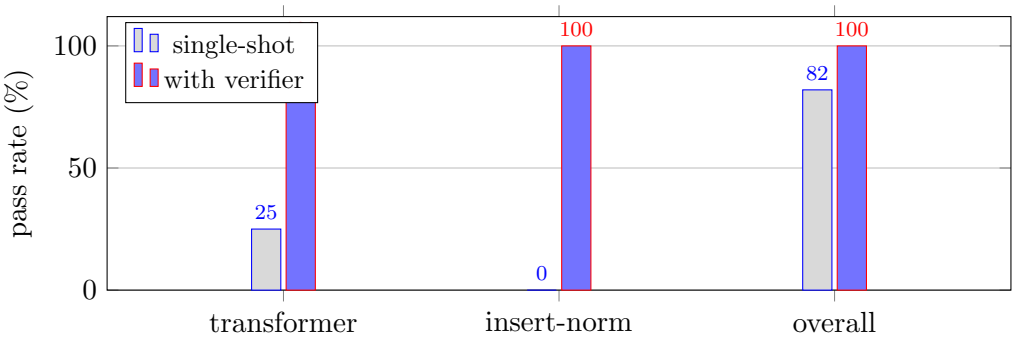

Figure 3: Verifier-in-the-loop lift for `claude-sonnet-4-6`. The overall gain (82 → 100) lands entirely on the two families a frontier model finds hard; the eight saturated families (all at 100%) are omitted for clarity.

LLM reward model	agreement with verifier	false-positive rate
(to be filled from `reward_anchor.mjs`)		

Table 4: An LLM reward model's agreement with the deterministic verifier, and its false-positive rate (approving a design the verifier proves is broken), per model. The verifier's own false-positive rate is 0% by construction.

design passes, its unsolved starting graph fails) and, given an LLM acting as a reward model, reports its agreement with the verifier and, crucially, its **false-positive rate**: how often it approves a design the verifier proves is broken. That number is what a lab needs to trust an LLM judge in this domain, and it can only be produced where a ground-truth verifier exists. This is a use of the environment beyond training or evaluation: a calibration anchor for the LLM reward models labs deploy where no verifier is available.

8 Related work

Verifiable agent benchmarks. SWE-Bench [1] and τ-bench [2] established the programmatic-verifier pattern in code and tool use; SWE-Gym [3] showed that a few thousand verifiable tasks suffice to train agents to a double-digit absolute improvement on SWE-Bench, and did so as free, open research. We target the same pattern in a domain that lacked a public verifier.

RL environments and verifiable rewards. Training on verifiable rewards (RLVR) is now central to reasoning-model post-training [4, 5], typically with policy-gradient methods such as PPO [7] and GRPO [6], preference optimization [9], or rejection sampling in the STaR [8] style. A growing market supplies labs with RL environments; reward-hacking resistance is the quality bar most consistently cited, and usable environments are expected to expose a hard band near a 2–3% pass floor. We build directly against both criteria, and ship GRPO and STaR loops against the same grader.

Neural architecture search. Classical NAS [10] automates architecture design under a proxy objective; our environment differs in providing a human-legible, per-edit, sub-millisecond verifier over a typed graph, and in serving as a training and evaluation substrate rather than a single search procedure. The accompanying product exposes a deterministic, constraint-satisfying design search (given a parameter budget, KV-cache budget, and target GPU, it returns the Pareto front over capacity, parameters, and KV cache), a NAS-lite special case built entirely from the verifier's

physics rather than a learned proxy objective.

9 Limitations

The transparent rubric released here does not verify linear-width consistency (Section 5); the health-score magnitude is not a quality ranking (Section 5); the reward path does not execute PyTorch per rollout, so shape-class failures are caught statistically rather than per-instance; and generated references lean on `replace_model` for design-from-spec families, biasing imitation style. Each is documented with its measurement or its planned mitigation in `ROBUSTNESS.md`.

10 Conclusion

Representing architectures as typed graphs turns "is this network sound, and what will it cost to serve" into a pure function. That function is simultaneously a leaderboard, an RL reward, and a verified-data generator, and it grounds against real PyTorch behavior on the claim that matters. We release the environment, the generator, the calibration and red-team harnesses, and the training loops, so that architecture design can join code and mathematics as a domain with a public verifier.

Reproducibility

```
git clone https://github.com/neurarch-ai/neurarch-arch-bench
cd neurarch-arch-bench
npx vitest run                              # satisfiability + non-vacuity + anti-gaming
node calibrate.mjs --policy=reference       # harness self-test (must be 100%)
node calibrate.mjs --policy=noop            # harness self-test (must be 0%)
node dump_grounding_set.mjs --count=40 --seed=123 --out=g.jsonl
python training/grounding_at_scale.py --set g.jsonl
python training/analyze_grounding.py
```

References

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*, 2024.

[2] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv:2406.12045*, 2024.

[3] J. Pan, X. Wang, G. Neubig, et al. Training Software Engineering Agents and Verifiers with SWE-Gym. *arXiv:2412.21139*, 2024.

[4] N. Lambert et al. Tulu 3: Pushing Frontiers in Open Language Model Post-Training. *arXiv:2411.15124*, 2024.

[5] DeepSeek-AI (D. Guo et al.). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948*, 2025.

[6] Z. Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv:2402.03300*, 2024.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*, 2017.

[8] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman. STaR: Bootstrapping Reasoning with Reasoning. *NeurIPS*, 2022.

[9] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct Preference Optimization. *NeurIPS*, 2023.

[10] T. Elsken, J. H. Metzen, and F. Hutter. Neural Architecture Search: A Survey. *JMLR*, 20(55):1–21, 2019.

[11] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS*, 2023.

[12] L. Roeder. Netron: Visualizer for neural network, deep learning and machine learning models. Software, https://github.com/lutzroeder/netron.